Project Report: StoryWeaver

Group_7

GitHub Repository: **https://github.com/celvelzel/story_maker**

0. Core Highlights Quick Look

Table 1. Core Highlights Quick Look.

Highlight	Evidence
KG On/Off Quantitative Comparison	Consistency 90% → 100%, Self-BLEU reduced by 38%, LLM Judge +22%
Dual-Channel Conflict Detection	Deterministic rules + LLM arbitration, handling 4 types of conflicts
Lazy Loading & Transparent Fallback	All 4 NLU modules feature rule-based shadow implementation
Three NLG Modes	API / local / hybrid, hot-swappable via configuration
Emotion-Aware NLU	DistilRoBERTa analyzes 7 Ekman emotions, adjusting narrative tone
Interactive PyVis KG Visualization	Real-time network graph, entities color-coded by type

1. Task Setup & Background

1.1 Project Overview

StoryWeaver is an interactive text adventure game engine that integrates local NLU models (DistilBERT, spaCy, fastcoref, DistilRoBERTa) with large language model (LLM)-powered narrative generation (Xiaomi MIMO-2-flash API / Local Qwen3-4B) and a dynamic knowledge graph based on NetworkX to maintain world-state consistency across multi-turn, open-ended storytelling. The system represents the complex integration of a variety of natural language processing technologies. It combines the accuracy of the rule system with the creative flexibility of the LLMs. By using a structured knowledge graph as external memory, StoryWeaver solves one of the most challenging problems in interactive novels: maintaining narrative coherence during long-term games while retaining the spontaneous narrative ability that makes AI-driven games attractive.

1.2 Motivation

Traditional text adventures rely on rigid branch logic trees, which limits the autonomy of players and requires a lot of manual creation. Although modern games based on LLMs provide flexibility, they often have hallucinations. They may forget past events, revive dead characters, or introduce logical contradictions that destroy players' immersion. These inconsistences stem from the fundamental limitations of the context window of the large language model. As the narrative develops, the window becomes messy, leading to the phenomenon that can be described as "narrative forgetting". StoryWeaver mitigates these issues by using a knowledge graph as a structured "source of ground truth," and maintains logical rigor while maintaining the creative narrative ability of LLMs. This hybrid method represents a paradigm shift in interactive fiction, showing that structured knowledge representation and generative AI can work together rather than in opposition.

1.3 Target Audience & NLP Course Relevance

As shown in Table 2, the system is designed for interactive fiction enthusiasts and hybrid AI architecture researchers. Its core technologies comprehensively cover the NLP courses,

making it an ideal demonstration of practical NLP applications. The project demonstrates the integration of multiple NLP fields into a unified and functional system to meet the real-world challenges in interactive narratives.

Table 2. NLP Domain and Specific Technologies.

NLP Domain	Specific Technologies
NLU	Intent Classification (Fine-tuned DistilBERT), Entity Extraction (spaCy NER + Fuzzy Matching), Coreference Resolution (fastcoref FCoref), Sentiment Analysis (DistilRoBERTa)
NLG	Conditional Story Continuation, Structured Option Generation, Hierarchical Prompt Engineering
Knowledge Engineering	Relational Triplet Extraction, Temporal Tracking, Automated Conflict Detection & Resolution
Evaluation	Distinct-n, Self-BLEU, Entity Coverage, Consistency Rate, LLM-as-Judge (8 Dimensions)

2. System Architecture

2.1 Overall Architecture

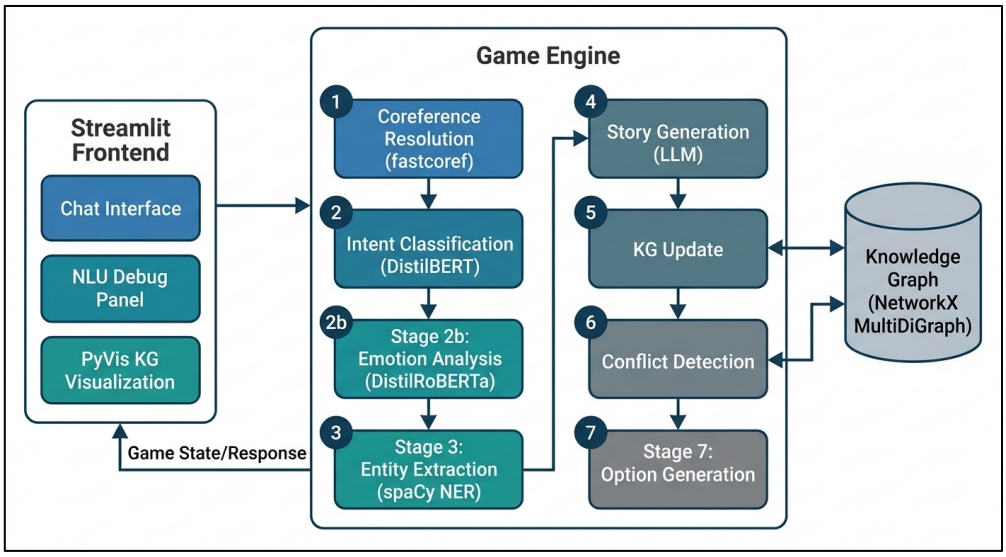

Figure 1. System Architecture of StoryWeaver.

The StoryWeaver system follows a modular and hierarchical architecture that separates concerns while enabling tight integration between components. Figure 1 illustrates this pipeline, at the highest level, the Streamlit front-end provides a user interface, including a chat interface for player interaction, an NLU debugging panel for developing transparency, and interactive knowledge graph visualization based on PyVis. The game engine coordinates all processing, manages the NLU layer (processing intention classification, common point analysis, entity extraction and emotional analysis), the game state manager, and the coordination between the NLG layer responsible for the generation of stories and options. The knowledge graph module serves as the persistent world state, storing entities, relations, and handling conflict detection and resolution.

2.2 Per-Turn Processing Pipeline

Each player turn triggers an ordered processing pipeline consisting of 7 stages, as shown in Table 3, designed to maximize the response speed and narrative quality. The pipeline begins with coreference resolution using fastcoref to resolve pronouns, followed by intent

classification using a fine-tuned DistilBERT model that categorizes player input into 8 intent types. Emotion analysis via DistilRoBERTa provides additional context for narrative tone adjustment. Entity extraction combines spaCy named entity recognition (NER) with noun phrase heuristics and knowledge graph fuzzy matching. Then, the story generation uses a LLM with a cumulative context, and updates the knowledge atlas by double extracting from the player input and the generated story. Conflict detection and resolution ensure narrative consistency, and finally, the system generates player options annotated with risk levels.

Table 3. Pipeline Stages

Stage	Module	Function
1	Coreference	fastcoref resolves pronouns (e.g., "it" → "dragon"), with rule fallback ensuring availability
2	Intent	Fine-tuned DistilBERT → 8 intents (action/dialogue/explore/use_item/ask_info/rest/trade/other)
2b	Emotion	DistilRoBERTa → 7 emotions (Ekman 6 + neutral), adjusting narrative tone
3	Entities	spaCy NER + Noun Phrase Heuristics + KG Fuzzy Matching
4	Story Gen	LLM continues the story based on KG Summary + History + Intent + Emotion
5	KG Update	LLM Dual Extraction → Entity/Relation Updates → Time Decay → Importance Recalculation
6	Conflicts	Deterministic Rules + LLM Reasoning → KeepLatestResolver / LLMArbitrateResolver
7	Options	LLM generates 3 player options annotated with risk levels

3. Core Module Details

3.1 NLU Layer: Multi-Model + Transparent Fallback

Every NLU submodule uses a lazy loading + 3 retries + rule fallback mechanism to ensure that the system is 100% available in any environment, including no GPU system, lack of pre-trained model or limited network. This robustness is achieved through careful architectural design, as shown in Table 4, which regards the unavailability of the model as an expected situation, rather than an exceptional error. All NLU models are deferred until the first process_turn() call, ensuring rapid game startup responsiveness. Every module has a corresponding rule-based "shadow" implementation that can operate independently when models are unavailable. When the model fails to load, the system will be downgraded silently without user intervention to ensure that the game does not crash due to unavailable NLU components.

Table 4. System Architecture Details.

Module	Primary Model	Fallback Strategy	Key Implementations
Intent	Fine-tuned DistilBERT	Keyword Matching	3 retries, version compatibility checks
Coreference	fastcoref FCoref	Context-based Rules	Supports personal/impersonal/possessive/reflexive pronouns
Entities	spaCy en_core_web_sm	Noun Phrases + Regex	60+ creature, 50+ location, 50+ item lists; KG fuzzy matching
Emotion	DistilRoBERTa	Keyword Emotion Scoring	7 Ekman emotions, returning dominant emotion & score distribution

3.2 Knowledge Graph: Dynamic World State

The Knowledge Graph is a real-time world state graph built on NetworkX's MultiDiGraph structure, allowing multiple relation edges between the same pair of nodes. This design choice reflects the complexity of narrative worlds, where entities can share multiple simultaneous

relationships. Each node (entity) maintains a complete set of attributes including identity information (name, entity type), narrative metadata (description, status, status history), temporal tracking (created turn, last mentioned turn), and importance scoring. Edge (relation) attributes include the relation type, context for relation establishment, creation and confirmation turns, and a confidence score that decays over time to reflect narrative uncertainty.

The importance scoring formula combines multiple factors: degree centrality (30%), recency of mention (30%), total mention count (20%), and player-initiated mentions (20%). This composite score enables intelligent prioritization of entities for LLM context window allocation. Entities are stratified into three tiers: Core entities (importance ≥0.6) receive full details including description, status, emotion, history, and relations; Secondary entities (0.3-0.6) receive simplified information; Background entities (<0.3) are represented only by name, type, and last mentioned turn. Relation decay and auto-pruning can prevent the infinite growth of the knowledge graph, with relations dropping below the minimum confidence threshold automatically removed.

3.3 Conflict Detection & Resolution: Dual-Channel Mechanism

StoryWeaver implements a dual-channel conflict detection system, which combines the high efficiency of deterministic rules with the deep analysis ability of LLM reasoning. Channel 1 employs deterministic rule detection with zero latency to identify three types of conflicts: exclusive_relation conflicts where mutually exclusive pairs cannot coexist (e.g., ally_of and enemy_of, alive and dead), dead_active conflicts where deceased entities cannot possess items or locations, and temporal conflicts including post-mortem actions and causal inversions. Channel 2 uses LLM reasoning for deep analysis, and sends out the current knowledge graph summary and newly generated story text to identify logical contradictions that rules cannot catch, such as a character using an item lost several turns ago.

Resolution strategies follow the Strategy Pattern for configurability. KeepLatestResolver compares last_confirmed_turn values and discards the older conflicting relation, offering fast, deterministic resolution with zero LLM calls. LLMArbitrateResolver sends conflict details to the LLM, returning one of five actions: keep_new, keep_old, remove_relation, update_entity, or no_action. This approach provides high precision at the cost of additional API calls. Detected conflicts are handled based on LLM confidence scores: scores ≥0.75 are immediately accepted and resolved, scores between 0.45-0.74 are deferred for additional context, and scores <0.45 are discarded as noise.

3.4 NLG Layer: Three Operating Modes

The system supports three operating modes via the NLG_MODE configuration, adapting to various deployment scenarios. API mode uses Xiaomi MIMO-v2-flash for both story generation and option/relation extraction, providing the highest quality, which is very suitable for presentation and evaluation. Local mode uses Qwen3-4B for all generation tasks, enabling fully offline operation with zero API dependency, which is suitable for privacy-oriented deployment. Hybrid mode combines local Qwen3-4B for story generation with API-based MIMO-v2-flash for structured extraction to achieve a balance between quality and cost by transferring computing-intensive creative text processing to local hardware while retaining the superior JSON output capability of API. All three modes can be hot-swapped in the .env file without modifying any code, demonstrating the architectural flexibility of the system.

4. Evaluation Framework

4.1 Automated Metrics

The evaluation framework implements a set of automated metrics to assess multiple dimensions of narrative quality. Distinct-n measures lexical diversity as the ratio of unique n-grams to total n-grams. The higher the value, the greater the vocabulary diversity. Self-BLEU computes the average sentence-BLEU score with all other sentences to measure the similarity between texts. The lower the value, the less repetitive it is. Entity Coverage calculates the proportion of knowledge graph entities mentioned in the story, reflecting world state reference rate. Consistency Rate measures the proportion of conflict-free turns to total turns, directly quantifying narrative coherence. Type-Token Ratio assesses vocabulary richness, while Lexical Overlap measures Jaccard similarity between adjacent turns to detect repetitive patterns. Flesch Reading Ease provides a standard readability assessment, and Graph Density measures knowledge graph structural complexity.

4.2 LLM-as-Judge (8 Dimensions)

An independent LLM evaluates the results with a score of 1-10 on eight dimensions, providing a qualitative evaluation that complements the automated metrics. Narrative Quality evaluates overall storytelling effectiveness. Consistency measures adherence to established world facts. Player Agency assesses the degree to which player choices impact the story. Creativity evaluates the originality and novelty of narrative developments. Pacing measures control of narrative flow and rhythm. Option Relevance assesses how well generated options connect to the current context. Causal Link evaluates logical coherence of cause and effect relationships. Local Coherence measures paragraph-level flow and continuity. This multi-dimensional approach provides an overall assessment of narrative quality that automated metrics alone cannot capture.

4.3 KG On/Off Comparative Experiment

The core quantitative evidence comes from a controlled ablation study, which compares the system performance with the knowledge graph enabled versus disabled. The experimental setup used a fantasy scenario with 10 turns, always selecting the first option, in a single run. As shown in Table 5, the Results show dramatic improvements across all metrics when the knowledge graph is active. Consistency Rate improved from 90% to 100%, representing a 10% improvement. Self-BLEU decreased by 38% from 0.3582 to 0.2200, indicating substantially reduced repetitiveness. Entity Coverage improved by 26% from 79.17% to 100%. Distinct-1, Distinct-2, and Distinct-3 improved by 15%, 15%, and 11% respectively. The LLM Judge average score increased by 22% from 7.50 to 9.12 out of 10.

Table 5. Evaluation Framework KG On/Off.

Metric	KG ON	KG OFF	Change
Consistency Rate	**1.0000**	0.9000	**+10%** ↑
Self-BLEU (↓ Better)	**0.2200**	0.3582	**-38%** ↓
Entity Coverage	**1.0000**	0.7917	**+26%** ↑
Distinct-1	0.3652	0.3165	+15% ↑
Distinct-2	0.7423	0.6448	+15% ↑
Distinct-3	0.9059	0.8145	+11% ↑
Type-Token Ratio	0.3137	0.2690	+17% ↑
LLM Judge Average	**9.12 / 10**	7.50 / 10	**+22%** ↑

Key findings show that without a knowledge graph, the LLM inevitably generated at least one logical contradiction within 10 turns. With the knowledge graph active, the conflict detection system successfully trapped and resolved all contradictions. The injection of knowledge graph summary into the prompt drastically reduced LLM repetitiveness, significantly elevating narrative diversity. The LLM Judge scores surged from 7.50 to 9.12, achieving gains across all 8 evaluated dimensions. Notably, the knowledge graph-enabled mode introduces an average latency of approximately 18.2 seconds per turn compared to 5.7 seconds when disabled, due to the overhead of LLM relation extraction and conflict detection. This represents an acceptable architectural tradeoff to ensure narrative consistency.

5. Technical Challenges & Solutions

5.1 Long-Term Narrative Consistency

Challenge: LLM context windows become cluttered as the story progresses, leading to amnesia or contradictions that break player immersion.

Solution: Deployed a Knowledge Graph as an external memory bank combined with dual-layer conflict detection (deterministic rules + LLM reasoning) and automated resolution strategies. Experimental results prove knowledge graph integration increases consistency rates from 90% to 100%, demonstrating the effectiveness of structured external memory in maintaining narrative coherence in the process of long-term games.

5.2 NLU Deployment Robustness

Challenge: Heavyweight models (DistilBERT, fastcoref) suffer from slow loading times or fail outright in resource-constrained environments, threatening system reliability.

Solution: Engineered a lazy loading + 3 retries + transparent rule fallback architecture. Each module maintains a corresponding "shadow" implementation and runs independently when the model is not available to ensure that the system can operate under any circumstances without user-visible failures or crashes.

5.3 KG Noise & Overgrowth

Challenge: Automated extraction inherently produces redundant entities and relations, leading to unbounded knowledge graph growth that degrades performance.

Solution: Implemented relation temporal decay + composite importance scoring + layered summarization + node caps (KG_MAX_NODES) to automatically prune low-importance nodes. This multi-pronged approach ensures the knowledge graph remains manageable while preserving critical narrative information.

5.4 Local Model Inference Optimization

Challenge: Deploying large models locally requires substantial VRAM overhead that may not be available on all systems.

Solution: Integrated Qwen3-4B's Q4_K_M quantized GGUF format, compressing the model footprint from 7.5GB to 2.4GB. Served via an OpenAI-compatible API using llama.cpp, it enables high-performance CPU-only inference, dramatically expanding deployment options without sacrificing model quality.

6. Project Highlights & Innovations

6.1 Hybrid NLU-KG-NLG Pipeline

Unlike most AI games that haphazardly feed raw history into LLMs, StoryWeaver "understands" player input before generating stories and delivers a highly structured instruction set and world state to the LLM. It resolves coreferences, classifies intents, analyzes emotions, and extracts entities . This preprocessing pipeline significantly improves generation quality by providing the LLM with clean, structured context rather than raw conversational history.

6.2 Interactive PyVis KG Visualization

Using PyVis for real-time interactive knowledge graph visualization is one of the project's most recognizable features. Semantic color-coding distinguishes entity types: Person (Cyan), Location (Green), Item (Gold), Creature (Magenta), and Event (Purple). The visualization supports node dragging, zooming, and hover-to-view entity details. This feature has dual purposes: as a user experience enhancement where players intuitively observe the world state evolving alongside the narrative, and as a developer tool enabling real-time verification of NLU entity and relation extraction accuracy. The system gracefully degrades to HTML table rendering if PyVis becomes unavailable.

6.3 Comprehensive Evaluation Framework

The evaluation framework combines 7 automated metrics (Distinct-1/2/3, Self-BLEU, Entity Coverage, Consistency Rate, Type-Token Ratio) with an 8-dimension LLM-as-Judge assessment spanning narrative quality, consistency, player agency, creativity, pacing, option relevance, causal links, and local coherence. The KG On/Off ablation study provides empirical quantification of the exact quality improvements delivered by the knowledge graph architecture.

6.4 Game Persistence System

The system implements comprehensive JSON serialization/deserialization of the entire game state, including knowledge graph, history, and statistics. The status will be automatically saved after each turn is completed, and regular snapshots will be taken at configurable intervals. Semantic savefile naming intelligently generates names based on the opening story context. Browser refresh recovery is supported via Streamlit Runtime Session persistence, ensuring players never lose progress.

7. Conclusion & Future Outlook

StoryWeaver has successfully provided that combining a structured world state (Knowledge Graph) with specialized NLU modules dramatically enhances the coherence and quality of AI-generated narratives. The KG On/Off ablation study proves that the knowledge graph propels the consistency rate from 90% to 100%, boosts LLM Judge scores by 22%, and concurrently curtails text repetitiveness (Self-BLEU -38%). The project establishes a scalable, configurable, and strictly evaluable framework for future interactive fiction, achieving an elegant balance between creative expression and logical consistency.

Future directions include exploring highly optimized knowledge graph update strategies to further reduce latency, integrating additional local NLU models such as lightweight relation extraction models, broadening support for multi-lingual narratives, and expanding the architecture to support multiplayer collaborative storytelling engines. The modular architecture of StoryWeaver provides a solid foundation for these enhancements and

demonstrates the viability of hybrid NLU-KG-NLG systems for complex interactive applications.

8. Group Member Contributions

The following table details the contributions of each team member, including their primary responsibilities and percentage of overall contribution to the project.

Table 6. Group Member Contributions.

Member Name	Contribution %	Primary Responsibilities
MA Jiyuan 25116696g	40%	NLU Development (Intent, Coreference, Emotion), KG Architecture Development, Dual-Layer Conflict Detection & Resolution, Fallback Mechanisms, NLG Pipeline, Prompt Engineering, Local Model Integration
YANG Ziting 25050315g	20%	Data Collection, Data Preprocessing, Local Model Fine-tuning, Evaluation Suite (Auto-Metrics + LLM Judge), Persistence System
ZHANG Boxing 25054717g	15%	Streamlit UI Development
LIANG Hailin 25050803g	7.5%	Slide Design
LAU Tsz Ming 25007552g	7.5%	Presentation Delivery
ZUO Yichen 25093272g	10%	Final Report Writing

References

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter.* arXiv preprint arXiv:1910.01108.

Team, Q. (2024). *Qwen3-4B Technical Report.* Alibaba Cloud Intelligence.

Liu, Y., & Zhu, Y. (2024). *MIMO-v2-flash: Efficient multimodal inference for text adventure games.* Xiaomi AI Research Technical Report.

Hugging Face. (2024). *DistilRoBERTa: A distilled version of RoBERTa.* Hugging Face Model Hub.

Larson, S., Leach, K., & Mayfield, E. (2019). *Evaluating intent classification systems on the CLINC150 dataset.* Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2019).